

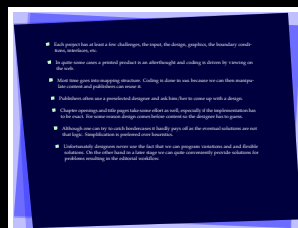
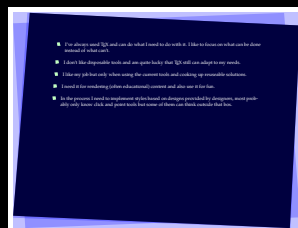
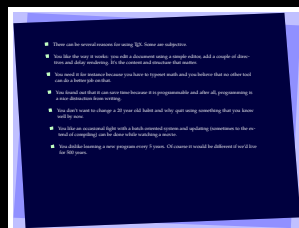
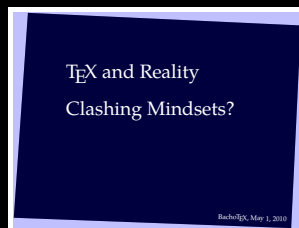
Presentations

This is a selection of three sets of presentations at meetings and a bunch of examples for some styles that come with **ConT_EXt**. In **MkIV** some of the older styles have been dropped. There are more presentations, starting mid 1990's: some are lost, others make no sense showing (or have articles instead) and I have no time anyway to collect all of them.

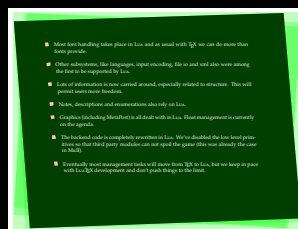
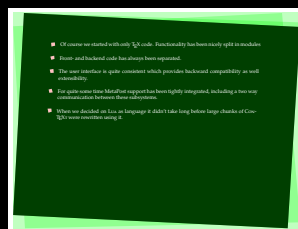
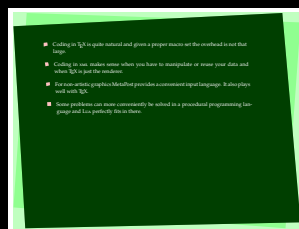
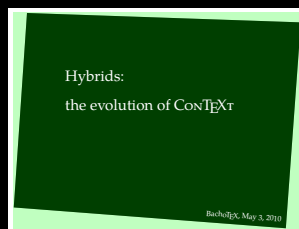
We show the first four pages but you can find the complete files in the **ConT_EXt** distribution. This file is generated automatically so it's also a snapshot. Maybe some day I find the time to add some more.

There are more files in the source tree because it doesn't make sense to provide all **pdf** files. Some files are to be generated with **LuaT_EX** because they discuss something (**MkIV**) that in the meantime has been superseded. Keep in mind that these talk are snapshots and might have presenting the state at that time, some experiments, or just have been teasers.

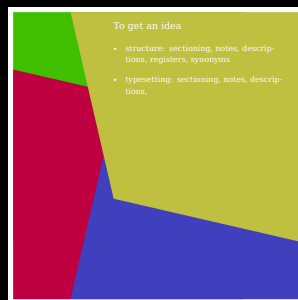
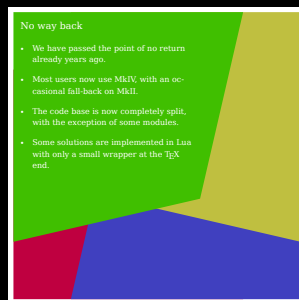
bachotex-2010-clash



bachotex-2010-move



bachotex-2011-cld-and-mkvi



bachotex-2013-bits

Bits and pieces:

ConTeXt
MetaPost
Lua and more

Hans Hagen
EuroBachOT_εX
May 2013

ConTeXt recently done	ConTeXt next on the todo list	MetaPost recently done
MetaPost next on the todo list	Lua recently done	Lua next on the todo list
LuaTeX recently done	LuaTeX next on the todo list	Fonts next on the todo list
Manuaals	Scripts	Speed

ConTeXt
recently done

- some more cleanup of old left-over
- most mechanisms now use the new level of parameter abstraction
- only a few fundamental incompatibilities (right of mechanisms, more control)
- slow introduction of hooks and extensions via setups
- first version of new multi-column routines
- replacing mechanism that have a (too) complex implementation
- exploring the mix (e.g. chemistry with Alan)
- remove some ugly left-overs from MetaPost math
- normalized dynamic fonts (possibly interfacing)

ConTeXt
next on the todo list

- more definitive split between generic and context (generated)
- some math extensions, maybe already math dictionaries
- normalize all styles and modes
- play a bit more with the Lua parser/loader
- add more integrated bib layout support
- finalize experimental (auto)script code
- investigate what more is needed in the cxx interface
- check multi-lingual interface translations (not entirely in sync now)

bachotex-2013-luatex

LuaTeX for dummies

(so you can still leave)

Hans Hagen
EuroBachOT_εX
May 2013

The TeX perspective	The Lua perspective	The hybrid perspective
The complications	The future	This workshop
Just plain	A bit less plain	Hardly plain

The TeX perspective

- it started out as pdfTeX
- then got merged with AlgaTeX
- but we left out the ugly bits of both
- its exclusively utf-8
- its math machinery got extended with OpenType like features
- there are no fundamental extensions as its impossible to agree in them

The LUA perspective

- it just a Lua engine
- it has some extra libraries on board
- you don't even have to use TeX
- but there are hooks into the TeX machinery
- and we can go further by loading libraries

bachotex-2013-sense

How about those
typographic virtues:
do they still make sense?

Hans Hagen
EuroBachOT_εX
May 2013

Typesetting	In the process	But eventually
Targeting paper	Towards displays	The state of affairs
Take ligatures	Accent battles	Justification
Backslashes	Endangered features	The future of TeX

Typesetting

- somehow we turned sounds into speech into language
- and after that it may have started with writing in the sand
- followed by painting on cave walls
- or maybe carving symbols in wood
- and figuring out some writing system
- that made it possible to chisel thoughts in stone
- and eventually putting blobs on paper
- that we somehow managed to map onto pixels
- but
- typesetting only happened very lately

In the process

- markup optimized the basic shapes (to suit the language)
- and kept inventing new symbols
- more and more automated typesetting and rendering and printing
- and of course this went with all kind of silly arguments
- think of this:
- for ages scribes were the only way to get high end arabic typesetting
- and now we can do this mostly automatic (in good and bad ways)
- so we endangered yet another craft

bachotex-2013-speed

BachoT_EX

Grandpa's toolbox

(making closets)

Dad's hobby

(cleaning closets)

Uncle's friends

(talking closets)

bachoT_EX 2016

–

+

- there's closets and closets
- take the ones you put stuff in ... e.g. that you buy at ikea
- you have to assemble them so you get out your toolbox
- this kid sits next to you wondering what that tool is
- what is natural for you to use might not be so for them
- but grandpa likes to carry over his knowledge and experiences

–

closets

+

- but ... kids get their information from the internet, not from you
- (they watch discovery channel or national geographic and know a lot)
- or they look at vloggers (no bloggers) trying to learn something
- and they keep moving on ... and on
- do gp's live long tools really make sense to them?

–

interest

+

- grandpa also has a vlogger, he is called knuth
- gp is as locked into tex as the kids are into apps
- he's a do-it-him-self kind of person
- will his grandkids love watching him crafting
- and hear his stories about meetings and journals

–

grandpas

+

bachotex-2017-emoji-demo

[illegible][illegible][illegible]

bachotex-2017-emoji

Picture Fonts

welcome to a (beautiful) mess

Hans Hagen
BachOTUG 2017

A Summary

- **the macro package's view:** just a font like any other but it needs to configure some extra color related properties
- **the engine's view:** depending on the technology a normal font that needs a bit special treatment or needs to be dealt with as collection of graphics
- **the viewer's view:** regular outline glyphs or images tagged as kind of characters so that their unicode representation can be cut and paste
- **the user's view:** more pictures than glyphs although some people one can communicate using them

So, in practice, for most $\text{\TeX}$ users it's probably not a high priority font but more a fun one.

Technologies

As each vendor came up with something, we have to deal with a all kinds of formats. And of course, as eagerness pushes things on the market before it's perfect we now have to deal with all of them.

- **overlapping glyphs:** this technique uses the colr and cpal tables and is actually a quite clean technology, you can combine in different ways
- **svg graphics:** this technique uses the svg table that contains a svg vector image
- **bitmap graphics:** this technique uses for instance sbix tables that can have various graphic images

The first two are already supported in the `CONTEXT` font loader and processor for a while, the last one was added recently.

Only the overlapping method is useable for the tens of thousands of skin tone combinations of families, (kissing) couples, and professions.

Preparation

For now one has to enable the feature:

```
\definefontfeature[overlay][default][ccmp=yes,colr=yes,dist=yes]
\definefontfeature[svg] [default][svg=yes]
\definefontfeature[bitmap] [default][sbix=yes]
```

```
\definefontfeature [colored] [default]
[cmcp=yes,dist=yes,
colr=yes,svg=yes,sbix=yes]
```

```
\definefont{MyEmojiFont} {seguiem}*overlay
\definefontsynonym{emoji} {seguiem}*overlay
```

As is using:

```
{\MyEmojiFont\resolvedemoji{woman}}
\emoji{woman}
```


Modern Latin

Why oh why

- In OnCtGh we have a mechanism to apply effects to a glyph stream.
- An active user on the OnCtGh mailing list wondered if that could be applied to specific fonts.
- The particular interest concerned the possibility to hidden fonts.
- I don't really like effects and not all fonts are suitable for it.

What are effects

```

FontEffect {
  inner
  outer
  both
}
FontEffect {
  inner
  outer
  both
}
FontEffect {
  inner
  outer
  both
}

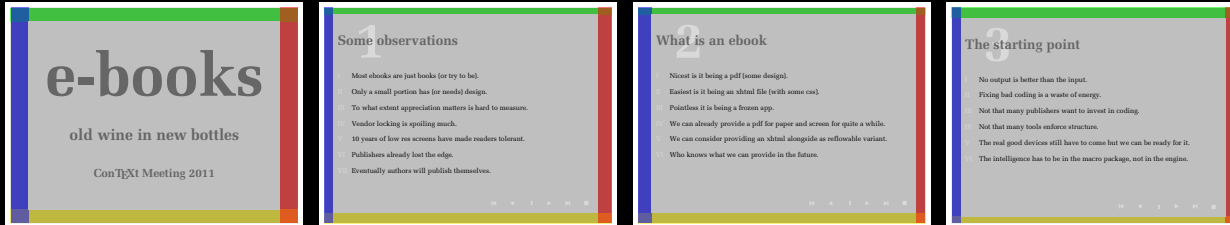
```

effect effect effect

inner outer both

[illegible][illegible]

context-2011-ebook-export



context-2011-mathml-update



context-2011-metapost-how-we-adapt



[illegible][illegible]

Breakers 2012	Existing sources	Breakers 2012	Existing sources	Breakers 2012	Existing sources
What is lexing - Computer languages have mandate structure. - You can avoid errors by checking the input. - Editors can help by coloring reserved words, concept etc. - Consistency in coloring different languages makes sense When did we start - We wrote our first editor begin 80's. - An extension, called Followed when we moved to the 90's called "L3" (later on quick down). - When MEGALA was no longer feasible we moved on to PRED, "L3" (later on quick down). - When we were done with the editor we were using the lexing. - I provided some guidelines for L3 and MEGALOG (support for multiple formats etc.).		Breakers 2012 Side effect of MKIV - SCITE got LEX based lexing (external lexing). - Later on I had already written some lexers for the pretty printers. - In 1996 it got me made some more advanced lexers. - These ship with CONQUEST, LEX, PRED, LIA, CLM, MEGALOG, test. Characteristics - The lex lower supports nested lexing of LIA and MEGALOG. - Lexical token (quick) checking is possible. - Unfortunately there is no lexing for SCITE on MEGALOG (but I wrote one more nowadays). - It is a pity that we have no access to SCITE internal as well with the original LIA interface. - On the good side lexing keeps the lexers somewhat close (at least).		Breakers 2012 The future - I will improve the current lexers. - An IDE, lexers will be added at some point. - I might make an IDE/LEX system that supports nested LIA.	

context-2012-mixed-columns

Breakdown 2012 The script

Output

- `ly` collects content paragraph wise.
- In between it can trigger the so called output routines.
- At last moment you can do something with the results.
- One of the things you can do is package all collected so far in a page.

Boxes

- In `lualatex` we can also intercept content at some places.
- For instance before and after each paragraph is processed.

Breakdown 2012 The script

Columns

- `ly` has no concept of columns.
- You need to fake them by filling with the width and spilling boxes.
- Often we run on `texlive` (no output routine).
- For some local applications we use simple columns.
- In the instance `lualatex` we used a mixed size and multi-column model.
- Columns are another (independent) mechanism, strongly grid based.
- Traditional multi-columns are being replaced by a new mechanism: mixed columns.

Pitfalls

- Footnotes: page, first or last column, each column (delayed, immediate).
- Graphics: mixing floats around is more complex than in single columns.
- Nesting: how about columns inside columns.
- Reducing: can be hard taking all into account.

Breakdown 2012 The script

Questions

- Do complex column mechanisms still make sense given the move to electronic paper?
- If so, what functionality should be provided.

context-2012-the-script

Breakdown 2012 The script

Some myths

- `CONTEXT` looks at lot like plain `tex` and expects users to program macros.
- `CONTEXT` depends on `TeX`.

The truth

- On the average users don't have to program. ConTeXting is not programming.
- As `ly` lacks conventional handling and job-control, helpers are provided.
- All context users can still program a lot, but not all need that.
- All context users can directly run `CONTEXT`, but why should they?

A few facts

- The `CONTEXT` distribution provides a sort of ecosystem.
- In `TeX` indeed we use `TeX` for some job control.
- But in `TeX` all is left context done in `TeX`.
- Two scripts play an important role: `mtxrun` and `context`.

Breakdown 2012 The script

The 'mtxrun' script

- Locates and runs scripts, has a lot of helpers provided.
- It is in fact the `TeX` runner on top the `ly` script.
- It knows about files and the environment we run in.
- It has some features that makes it easier to integrate in services.
- The way we don't need much (and avoid potential conflicts in some).

The 'context' script

- It runs `CONTEXT` and keeps track of how many runs are needed.
- Contrary to its `TeX` ancestor it is not needed for make testing etc.
- It has a few extensions that are loaded on demand: `extrat`.

Breakdown 2012 The script

A regular run

```
context [-:run] filename
```

Running from an editor

```
context --script filename
```

Running from an service

```
mtxrun --path=filepath -script context filename
```

Breakdown 2012 The script

Controlling the rendering

```
--noheaderlist  
--nofooterlist  
--noindexlist  
--noimageindexlist  
--noindexlist
```

Controlling with `ctx` files

```
--ctxname
```

Also in preamble

```
<!context-directive job stylefile email.ctx >
```

context-2012-visual-debugging

Breakdown 2012 Visual debugging

How it started

- Some 15 years ago I wanted some more feedback.
- First I started out a way to translate boxes, lines, etc.
- Some aspects were tricky, like stretch and shrink (as a `TeX` pre), filters, leaders, etc.
- I gave some presentations and it was not to see the parallel faces.
- As debugging didn't work, it is somewhat interfering.
- When not needed there is no overhead but we still disable it at some places.

Do we need it

- I wonder if anyone ever used it.
- Some of the helpers are quite handy, like `lyvisual`.
- So there had to be provided anyway, so: when to stop?

Breakdown 2012 Visual debugging

All kind of debugging

- We have some debugging, much shows up when writing new code.
- Think of fonts, math, graphics, characters, etc.
- Some make us aware in `TeX`, so they're gone, but some more show up.
- In that time there will all be surrounded (or more) lines in modules.

Breakdown 2012 Visual debugging

Why we kept it

- When cleaning up the code I had to decide to keep it or redo it as it could be done `TeX`-ish.
- But as we already had some `TeX` based extra it made sense to redo it.
- The old code is still there as module (also because it had some more useful).

How it worked

- In `TeX` procedures are overloaded.
- So effectively when enabled, `TeX` can run become macros.
- We use rules (and leaders) to window properties.
- Some constructs interfere so we need to compensate side effects.

Breakdown 2012 Visual debugging

How it works

- The helper were a rather trivial quick job as we had a lot in place already.
- Interpreting the mode list and existing structures.
- The new colors, rules and text box stuff can be overlaid.
- Control over what gets rendered at the `TeX` end.
- Control over what gets shown by using layers.
- A second most time more into mechanisms, choices and optimization.
- Some structures interfered with (hardcoded) expectations in the backend.
- When I decided to run layers I had to adapt some other code (mostly out of efficiency).
- There is room for more (but first I want the bits of `lualatex`).

context-2012-xml-news

Breakouts 2012	Prereleasing XML, some basics
Topics	
• processing	
• selecting	
• flushing	
• testing	
• basic's only	

```

Processing XMI, some basics

Processing
\winprocess\file (name) (filename) (setup)
\winprocess\buffer (name) (filename) (setup)
\winload\ (name) (filename) (setup)

Loading
\winload (name) (filename) (setup)
\winload\buffer (name) (buffername) (setup)

```

```

Injecting elements

\nu1val1 (mode) (pattern)
\nu1first (mode) (pattern)
\nu1last (mode) (pattern)
\nu1fn (mode)
\nu1raw (mode) (pattern)
\nu1content (mode) (pattern)
\nu1attr (mode) (pattern)
\nu1tag (mode)
\nu1text (mode) (pattern)

```

```
Bash: $ cd ..
```

```
Processing XML... some.html
```

```
Injecting attributes
```

```
\setatt      (node) (name)
```

```
\setattf     (node) (name) [default]
```

```
\setattribute (node) {pattern} (name)
```

```
\setattifnode (node) {pattern} (name) (default)
```

```
Injecting properties
```

```
\setcount   (node) (pattern)
```

```
\setname    (node)
```

```
\setnamespace (node)
```

context-2013-math

Math:

progress or standing still

Hans Hagen
ConTeXt Meeting
September 2013

Math as script	Alphabets	Heavy bold	Radicals
Primes	Accents	Stackers	Fences
Directions	Structure	Italic correction	Big
Macros	Unscripting	Combining fonts	Tracing

[illegible]

Alphabets

- the shape (style) of a character determines its meaning
- but in most cases an type is embodied in *one* character
- and tagged with some rendering directive, often indicating a font style
- in traditional T_gg we have alphabets in different fonts, so we're talking switches
- in TeXcon and OverTeX we have alphabets with standardized code points (but *gaps too*)
- this has big advantages for communicating, transferring data etc.
- but a math engine still has to deal with *sax* input as well
- multiple axis: types, alphabets, styles, variants, shapes, modifiers

We're off better but the gaps are an annoyance

context-2013-speed

Speed:

why it matters
and why we care

Hans Hagen
ConTeXt Meeting
September 2013

Speed	Pages per minute
What happens	What we can do

Speed

- speed matters in a edit-run-preview cycle although this is mostly perception
- the more the interface, the slower it goes, but you seldom set something up so that it is not much of a burden
- everything you provide gets used at some point, also in sufficient ways, so best know your weak spots
- lots of local (grouped) tweaks leads to many mechanisms lacking in unseen, grouping matters
- writing use of functionality can have drastic and unexpected speed penalties

Pages per minute

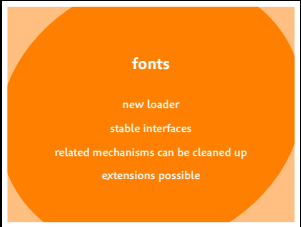
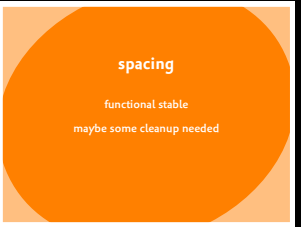
- we set up the baseline performance (in pages per second) as much as possible
- we try to identify and optimize critical routines, both at the `TeX` and `LaTeX` end
- we compare the used hardware machine and version of `LaTeX` and `CiTeX` matter

Microtime (10000 'count' '\pages')

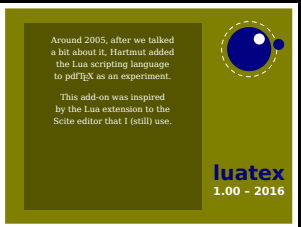
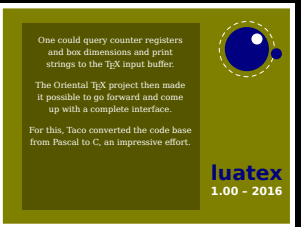
Page	Jan	Apr	May	May	init
1	2	2	2	4	4
10	15	17	17	36	37
20	50	109	110	217	236
1000	185	234	219	509	512
10000	2315	298	284	638	627

2/98/2013, Lua^{TeX} 0.722, Dm6 6700, SSD, 16GB, 2.80 GHz 3840QMU, Windows 8 /32 bit
 06/20/2013, Lua^{TeX} 0.722, Dm6 6700, SSD, 16GB, 2.80 GHz 3840QMU, Windows 8 /64 bit

context-2015-status

	 fonts new loader stable interfaces related mechanisms can be cleaned up extensions possible	 hyphenation experimental normalization considered	 spacing functional stable maybe some cleanup needed
--	--	---	--

context-2016-luatex

 LuaTeX Version 1.00 ConTeXt meeting – September 2016	 After ten years of stepwise development and experimenting we release version 1.00 of LuaTeX during the 10th ConTeXt meeting in the Netherlands, September 2016. The interface is now rather stable and will not change significantly which means that one can write stable packages. So, it's time for a bit reflection as well as time to tell what we will be doing next. luatex 1.00 - 2016	 Around 2005, after we talked a bit about it, Hartmut added the Lua scripting language to pdfTeX as an experiment. This add-on was inspired by the Lua extension to the Scite editor that I (still) use. luatex 1.00 - 2016	 One could query counter registers and box dimensions and print strings to the TeX input buffer. The Oriental TeX project then made it possible to go forward and come up with a complete interface. For this, Taco converted the code base from Pascal to C, an impressive effort. luatex 1.00 - 2016
---	--	---	--

context-2017-css-selectors

CSS selectors

- used in HTML styling
- a bit different from the path based method
- shortcuts for filtering by attribute
- class filtering is special because it checks for list
- one can select more at the same time
- performance is okay compared to path lookup
- selectors go between curly braces

```
selectAll (#) {foo bar .whatever, bar foo .whatever}}
```

The following methods are supported:	
<code>element</code>	all tags element
<code>element -> element</code>	all tags element with parent tag element
<code>element + element</code>	all tags element + generated by tag element
<code>element - element</code>	all tags element - generated by tag element
<code>element * element</code>	all tags element * inside tag element
<code>attribute</code>	all attributes
<code>[attribute]</code>	attribute spans value
<code>[attribute="value"]</code>	attribute contains value (space is separator)
<code>[attribute="value"]</code>	attribute starts with value
<code>[attribute="value"]</code>	attribute ends with value
<code>[attribute="value"]</code>	attribute contains value
<code>id</code>	has id
<code>#id</code>	has class
<code>child-class</code>	the child of node x
<code>next-last-child</code>	the last child from the end
<code>first-child</code>	the first child
<code>last-child</code>	the last child
<code>match at index</code>	the match at index x
<code>next-last-of-type</code>	the last of type from the end
<code>first-of-type</code>	the first match
<code>last-of-type</code>	the last match
<code>only-of-type</code>	the only match or nothing
<code>not-child</code>	the only child or nothing
<code>empty</code>	only empty
<code>!not</code>	the whole tree

[illegible]

context-2017-features-chaintest



context-2017-features-kerntest



context-2017-features-pairtest

A B C D E F . . .

context-2017-features-singletest

A B C D E F . . .

context-2017-features-spacetest

A B C D E F G H . . .

context-2017-features-substitutiontest

abracadabra
XPrXcXdXPrX
YQrYcYdYQrY
XYPQrXYcXYdXYPQrXY
1raca2bra
aBraCadaBra
effe 1b3def !f . . .

context-2017-features

Font features

What are they
Substitution
Positioning
Ligatures
Pseudo
Examples

CONT_EXt 2017 Mailbach

- built in substitution that is often optional like ligatures but for some languages mandatory
- built in positioning that is assumed to be applied like kerning, mark anchoring, etc.
- external properties like coloring, spacing, fallback combinations
- engine related tricks like expansion and protection
- tracing options
- whatever you like ... so let me know
- so in ConT_EXt we have font features (driven by font) and pseudo features (driven by additional needs)

What are they

- single: replace one by another
- alternate: replace one by one of a set
- multiple: replace one by multiple others
- ligature: replace multiple by one shape
- contextual lookups and replacements with look back and look ahead

Substitution

- single: repositioning a glyph (with optional marks), this includes traditional kerning
- pairwise: repositioning two adjacent glyphs (with optional marks)
- anchoring: often used for marks to base glyphs, ligatures and other marks
- attachment: often used for various scripts, putting glyphs in a word together
- contextual lookups and positioning with look back and look ahead

Positioning

context-2017-performance

Performance

Why bother
Possible bottlenecks
Measurements
Examples

CONT_EXt 2017 Mailbach

- because we don't want to waste time waiting
- because I get tired of ignorance related complaints
- because it (sometimes) can be a nice puzzle to improve performance
- because it occasionally reviewing makes code better

Why bother

- starting up
- loading fonts
- processing features
- applying trickery
- enabling tracing
- fonts
- L^AT_EX
- images

Possible bottlenecks

- how long does a run take
- how does the number of pages matter
- how many runs are needed
- start-up time
- processing pages
- finishing the document
- the console used

Measurements

context-2017-synctex

SYNCTEX

What is SYNCTEX
Disclaimer
What we do
How it works
What the user gets

CONT_EXt 2017 Mailbach

- it is a mechanism for going back from viewer to editor
- it uses an extra (cached) output file
- it adds an overhead of 2 to 15 percent runtime
- it is designed with a specific macro package in mind
- the rather generic approach works okay for simple document layouts
- but it often fails for projects that use multiple files
- and that moves information around like X_YL recorded files

What is SYNCTEX

- all recently it was supported in ConT_EXt as it was
- there were no complaints, so it must have worked ok for most users
- we never used it ourselves because of mentioned reasons
- we only wanted to support it when it works ok in projects
- (think of thousands of X_YL with deeply nested inclusions in one document)
- but what we support now is purely based on personal experience
- we don't use it ourselves so feedback is welcome

Disclaimer

- the internal SYNCTEX mechanism is disabled
- when told so, ConT_EXt will look to its own code
- this is done by using L^AT_EX code to set the right information
- only source files that make sense are dealt with
- this protects the styles from unwanted changes
- within reasonable bounds X_YL is supported
- this also includes nested documents

What we do

The diagram illustrates the relationship between four LaTeX table packages: **Tables**, **Variants**, **TABLJ**, and **Tabulate**. Each package is represented by a box containing a list of features or characteristics. Arrows indicate dependencies or relationships between the packages.

- Tables** (leftmost box):
 - Variables
 - TABLJ
 - Tabulate
 - Natural tables
 - Extreme tables
 - Long tables
 - Frame tables
- Variants** (second box from left):
 - Good old TABJ = wrapper about T_{AB}J's alignment.
 - Running text, break across pages tabulates.
 - Tables that behave like HTML tables therefore called natural tables.
 - A variant on this that is more easy to extend, tagged extreme tables.
 - A low profile installable mechanism that can open pages and breaks well.
 - A way to make huge tables without overflowing T_{AB}J too soon.
- TABLJ** (third box from left):
 - Based on the T_{AB}J macro package.
 - Detailed control over spacing.
 - Somewhat inconsistent spacing out of the box.
 - Extended with extra features.
 - Mostly rewritten but within the original concept.
- Tabulate** (rightmost box):
 - Mostly meant for tables that are part of the text flow.
 - Breaks paragraphs across pages.
 - Sort of compatible in control with T_{AB}J.
 - The system that I used most often.
 - It uses multiple passes if needed.

Lean and mean

- interference:** LuaTeX, plainTeX and CTAN all have different demands (we want to experiment and more we can control path by way)
- consistency:** the source tree is way too complex as in the build (you only need LuaTeX)
- distributions:** no one can guarantee stability for LuaTeX (there is a more player that offers it the other)
- annotations:** experimental code leads to bugs outside CTAN and that triggers complaints
- modernity:** running into files who know to express "more logic" and "more modularity" makes sense
- gameplay:** I got tired of "you need to express this because..." habits
- possibilities:** we have the better separation between the LuaTeX driver and output format drivers

ConTeXt meeting, September 2019

Hans & Alan

LuaMetaTeX
How it became

What it is

- simplification:** we don't need all what is currently in the LuaTeX source (we don't need it)
- source:** there is much less of it and we can get rid of the stuff we don't need
- compilation:** there was much more stuff on the way needed and only a few know these details
- consistency:** to guarantee consistency with LuaTeX the source code will be part of the source distributions (even I'm satisfied)
- marketing:** the way the relation with LuaTeX and its source code will be seen clear
- gameplay:** we have more relevant and important the playing of coming into relation with more CTAN users... "we did it now we did it"
- possibilities:** the playing of more with the late and possibilities that are responsible for most interesting issues, like the (un)permitted page layout

LuaMetaTeX
What it is

- history:** there is only one relatively small history (that does not change much)
- code base:** there exists an extra source tree, but it's not a separate version of LaTeX
- user control:** if needed users can compile the program as we will be contained
- future safe:** we can move forward and improve
- modernity:** a code base with the latest LaTeX, english and latex
- side effect:** we drop LuaTeX as it doesn't keep up (and results are the same)
- design:** we have a better separation between the LuaTeX driver and output format drivers (between the LuaTeX driver and the output format drivers)
- evolution:** there is no need to change the behavior (in source, we keep it all used in the code base (we only use a few small third party libraries))

LuaMetaTeX
Implications

[illegible]

ConT_FXt Meeting

context-2020-datatypes

DATATYPES

additional datatypes in lmtx

Native TeX datatypes: simple registers

```
Integer: %count 123 + 456 %\the\count123
Integer: 456
Dimension: %dimension123 + 456pt %\the\dimension123
Dimension: 456.0pt
glue: %skip123 + 4pt plus 5pt minus 4pt\relax %\the\skip123
glue: 0.0pt plus 5.0pt minus 4.0pt
margin: %margin123 + 5cm plus 5cm minus 4cm\relax %\the\margin123
margin: 6.0cm plus 5.0cm minus 4.0cm
attribute: %attribute123 + 456 %\the\attribute123
attribute: 456

\global %\the\count\dimension\skip\margin\attribute
```

```
\advance \multiply \divide \numexpr \dinxpr \glueexpr \muexpr
```

100

Native TeX datatypes: tokens

```
toks: \toks123 = {456} \the\toks123
toks: 456

\global \the \toksdef
\toksapp \toksapp \toksapp \toksapp
\tokspre \tokspre \tokspre \tokspre
\im retrosp: \endx
```

context-2020-ecmascript

ECMAScript

just because it can be done

Why oh why

- We use a *mu2fi* based pdf viewer: *SumatPDF*.
- And occasionally we use the tools that come with *mu2fi*.
- So when checking if that viewer supports JavaScript in widgets I noticed the stand alone interpreter:¹
- Which made me wonder how easy it would be to interface to it.
- It uses the lightweight library subsystem: like *lib* the library interface is setup dynamically.
- Support is not integrated in *LuaMetaNGC*, so there is no overhead and there are no dependencies.
- We assume that the library is on the system, and when not, then there is also also no support.
- We stick to the absolute minimum of interfacing needed and delegate everything else to *Lua*.
- We assume a stable api, and if not, well ... sorry.

¹ The official name is *ecma-script* which is the standardized core language.

The components

- The optional, delayed loading, interface, adds only a few KB to `LibMata20x`.
- The `Libm` library interface that is part of the `ComT20` distribution which means that it's officially supported.
- There is a `T20` module that loads the lot and provides the user interface.
- And of course, somewhere on the system, there should be the `cu32` library.²
- A module like this should conform to the `ComT20 LINTX` standards (a minimalistic not bloated api, interfacing in this should be `void` etc.).

In ConTeXt libraries go into the platform tree, like

```
/tex/texmf-win64/bin/lib/luametatex/mujs/libmujs.dll
/tex/texmf-linux-64/bin/lib/luametatex/mujs/libmujs.so
```

² Taim compiled the library for his system during the talk and confirmed that it also works out of the box on an

eemancip - context 2020 meeting - the composition

An example

```
%usemodule([xmacscript])

%macrocode {
  comesc1e(""); comesc1e("when the loading has succeeded!");
  comesc1e("");
}

%macrocode {texprint{"Just a [\\bf short] sentence."}}

%startmacrocode
  %texprint{"And this is \\inframed{\\bb a bit longer} sentence."}
%stopmacrocode

Just a short sentence.

And this is a bit longer sentence.
```

context-2020-implementers

IMPLEMENTERS

an old feature still evolving

context 2020 meeting

Interfacing with Lua

- Quite some activity is delegated to Lua.
- Normally the initiative is at the $\mathcal{V}_{\text{LUA}}$ end.
- We can set variables or call functions etc.
- We can parameterize to function calls.
- From the Lua end we can use scanners to pick up data.
- We provide some consistent interfaces for doing all that.
- From $\mathcal{V}_{\text{LUA}}$ to Lua we use $\text{setLua}\{\dots\}$ and friends.
- From Lua to $\mathcal{V}_{\text{LUA}}$ we use $\text{context}(\dots)$ and alike.
- For adding functionality we use so-called implementers.

implementers – context 2020 meeting – interfacing with bus

Calling Lua

```

letValue[context("ok")]
ok
letValue[context(2 + tokens.scanners.integer())] 20
20
startLineEnd
function document.MyThing() context(2 + tokens.scanners.integer()) end
stopLineEnd
letValue[document.MyThing()] 20 \end
letValue[document.MyThing()] 30 \end
letValue[document.MyThing()] 40
ok ok ok

```

implementary – content 2020 meeting – calling for

Streamlining Lua

```

\startliteratecode
interface implement {
  name = "MyThing",
  public = true,
  arguments = "integer",
  actions = function(s) context(s + 2) end,
  -- actions = { function(s) return s + 2 end, context }
}
\stopliteratecode

MyThing 20 Iqued MyThing 20 Iqued MyThing 40
40 40 80

```

implementing – context 2020 meeting – streamlining bus

LUAMETATEX
where do we stand
context 2020 meeting

baometatex -- context 2020 meeting -- when it started

luametaux - context 2020 meeting - why it started

huametatex - content 2020 meeting - the development

MKII MKIV LMTX
 where does it end
 context 2020 meeting

METAFUN
simple fonts
context 2020 meeting

ConT_FXt Meeting

SVG GRAPHICS

some demos and discussion

context 2020 meeting

- It expands to fit Scribble Vector Graphics.
- It is an example of application and format mismatch.
- It varied our sample kind of expanded Picturing to save format.
- It took a while to be pushed up as output format.
- It prevents you get the same money build-up as in other vector formats.
- This is a side effect of other constrained editing.

© Microsoft/Adobe give a demonstration of editing in InDesign and there was some discussion about this aspect

erg graphics ———— 2002/02/ meeting ———— werg

- Properties can be set as attributes to an element (any value).
- Properties can be set in the `<XML>` namespace separated (any value).
- Properties can be set via one of many `XML` assignments.
- Properties can be bound to a specific element
- Properties can be inherited from an ancestor (sometimes vague).
- Properties can be redefined (locally, overwritten (parent, style), values can add their own, etc.
- ... it's kind of a mess.

[illegible]

TOKENS

tokens as I see them

context 2020 meeting

About tokens

- Like nodes, it's a common term used in programming.
- In TSP, The Program tokens and nodes are therefore count present.
- For most cases they are irrelevant concepts.
- We can well explain them using memory.
- Let's try to avoid the establish token-speak sometimes heard in the community.
- So... I won't correct you as long as you don't correct me.
- Let's now enter the world of tokens in the naive way.

What are tokens

- It is an internal data structure, effectively a 32 bit integer
- This integer encodes a constant (operand) and an extra code (operator).
- But often it's not a character but more a **NaN** command.
- Represent a converted token values
- Values are either expanded (integer/float or array).
- When they are stored they are part of a larger data structure, a memory word.
- Token memory is an array of many memory words.
- The token memory 'word' has two integers: a token value and an index into token memory.
- The type 'tag' can have forward linked list of tokens.
- A hash table maps control sequences into indices into token memory.

tokens → control 2020 something → what are tokens

- Sometimes there is special hand when at the start.
- A hand takes much for easier representing of events count.
- Elements from the last side for a given token.
- Original γ does global temporary lists.
- This is needed when we expand (insert) and want to report issues.
- This is not needed when we just normalize (which we do a lot in $\text{Lau}^{\text{Lau}}_{\text{Lau}}$).
- So, this is all optimized for performance and memory consumption.
- Pseud lists are collected in a cache so tokens can get scattered.
- In $\text{Lau}^{\text{Lau}}_{\text{Lau}}$ they may as close to original γ as possible.
- But the Lau interface forces us to occasionally store.

COMPACT FONTS

a different approach to scaling

context 2021 meeting

How `TeX` uses fonts

- In a `TeX` engine there is a 1-to-1 mapping from characters in the input to a slot in a font.
- Combinations of characters can be mapped onto one single ligature.
- Hyphenation is determined by the upper boundary of font which requires some facilities.
- In the typical result characters might be implemented relatively barren.
- That all happens in a very intermittent way (e.g. inside the `par` holders).

compact fonts → current `TeX` setting → how to use fonts

How traditional TeX sees fonts

- A font is just a binary blob of data with information where characters have a width, height and descender.
- The engine only knows the dimensions and when the same font is used with a different scale a copy with scaled dimensions is used.
- There can be requests for figures and sizes (complex) so called math extensible shapes (like arrows, with accents, brackets and italics).
- There are pairwise specifications for kerning.
- A handful of font parameters that are provided in order to control for fontface spacing.
- Verbal font face names normal fonts but there have been requests for the backend to convert shapes TeX uses the normal metric file, the backend also the verbal specification file.

An example of a definition and usage:

```
\font\myfont {source1a} test [x test] test
```

source1a font → source 1021 metric → how traditional TeX sees fonts

Unicore engines

- There are no fundamental differences between a traditional engine and an Unicore aware engine with respect to their 54 words from kernel dimensions.
- There can be more than 256 characters in a font.
- Some mechanisms have to be provided for applying font features (like ligaturing and kerning).
- In Lua², hyphenation, ligaturing and kerning are clearly separate steps.
- In Lua², calligraphs can do lots of things with fonts and characters.
- Math fonts are handled differently and there are more tools that concern from the font.
- Math fonts handle this signature are implemented differently. i.e. no spaces of fonts are needed and applied separately beside with the glyphs.

compact fonts – emulated 321 modeling – unicore engines

PROGRAMMING
local control
context 2021 meeting

- Can't be in several call reading from reading module:
 - there must only be reading from the source files. Characters are picked up and interpreted. Depending on what takes it becomes some action like plus
- 1 `\\various\\dots to \\input{2}` `\\various{}` the count `\\multitply{count by 4}`
- The 2 gets triggered because characters like the ones on text
- The `\\various{}` primitive triggers picking up a regular number, then goes on scanning for a bus (separator) and that itself will trigger a sequence if whatever until the group ends
- The count primitive triggers scanning for a regular number (or reference) and then scans for a number: the output sign is optional.
- The primitive injects some value into the current input stream (if does so by entering a new input)
- `\\input{2}` triggers picking up a regular sequence and multiplies that by the next scanned number. The by is optional.
- Printing from LaTeX and scanning (things with e.g. `\\various{}` in LaTeX reading (pseudo) files.

programming – context 2021 meeting – expansion

```

%valof TestA 1 {valofdb {hsho(2) {count=3 {the{count}}
%valof TestID(1) {valofdb {hsho(2) {count=3 {the{count}}

```

TestA				TestID			
id	val	db	hsho	id	val	db	hsho
TestA001	1.0	1	1	TestID001	1.0	1	1
TestA002	2.0	1	2	TestID002	2.0	1	2
TestA003	3.0	1	3	TestID003	3.0	1	3
TestA004	4.0	1	4	TestID004	4.0	1	4
TestA005	5.0	1	5	TestID005	5.0	1	5
TestA006	6.0	1	6	TestID006	6.0	1	6
TestA007	7.0	1	7	TestID007	7.0	1	7
TestA008	8.0	1	8	TestID008	8.0	1	8
TestA009	9.0	1	9	TestID009	9.0	1	9
TestA010	10.0	1	10	TestID010	10.0	1	10
TestA011	11.0	1	11	TestID011	11.0	1	11
TestA012	12.0	1	12	TestID012	12.0	1	12
TestA013	13.0	1	13	TestID013	13.0	1	13
TestA014	14.0	1	14	TestID014	14.0	1	14
TestA015	15.0	1	15	TestID015	15.0	1	15
TestA016	16.0	1	16	TestID016	16.0	1	16
TestA017	17.0	1	17	TestID017	17.0	1	17
TestA018	18.0	1	18	TestID018	18.0	1	18
TestA019	19.0	1	19	TestID019	19.0	1	19
TestA020	20.0	1	20	TestID020	20.0	1	20
TestA021	21.0	1	21	TestID021	21.0	1	21
TestA022	22.0	1	22	TestID022	22.0	1	22
TestA023	23.0	1	23	TestID023	23.0	1	23
TestA024	24.0	1	24	TestID024	24.0	1	24
TestA025	25.0	1	25	TestID025	25.0	1	25
TestA026	26.0	1	26	TestID026	26.0	1	26
TestA027	27.0	1	27	TestID027	27.0	1	27
TestA028	28.0	1	28	TestID028	28.0	1	28
TestA029	29.0	1	29	TestID029	29.0	1	29
TestA030	30.0	1	30	TestID030	30.0	1	30
TestA031	31.0	1	31	TestID031	31.0	1	31
TestA032	32.0	1	32	TestID032	32.0	1	32
TestA033	33.0	1	33	TestID033	33.0	1	33
TestA034	34.0	1	34	TestID034	34.0	1	34
TestA035	35.0	1	35	TestID035	35.0	1	35
TestA036	36.0	1	36	TestID036	36.0	1	36
TestA037	37.0	1	37	TestID037	37.0	1	37
TestA038	38.0	1	38	TestID038	38.0	1	38
TestA039	39.0	1	39	TestID039	39.0	1	39
TestA040	40.0	1	40	TestID040	40.0	1	40
TestA041	41.0	1	41	TestID041	41.0	1	41
TestA042	42.0	1	42	TestID042	42.0	1	42
TestA043	43.0	1	43	TestID043	43.0	1	43
TestA044	44.0	1	44	TestID044	44.0	1	44
TestA045	45.0	1	45	TestID045	45.0	1	45
TestA046	46.0	1	46	TestID046	46.0	1	46
TestA047	47.0	1	47	TestID047	47.0	1	47
TestA048	48.0	1	48	TestID048	48.0	1	48
TestA049	49.0	1	49	TestID049	49.0	1	49
TestA050	50.0	1	50	TestID050	50.0	1	50
Test							

[illegible]

LUAMETAFUN
the new interfaces
context 2021 meeting

- The core of the engine is still `TyE`. It all starts there.
- The `TyE` instances are opened up to `Las`. We can call out to `Las` from `TyE` and to some extent from `Las` to `TyE`. Quite often we push back something using the input channels.
- The MetaSML language is accessed from `Las`. So from within `TyE` there is always `Las` in between. Being able to reach out to `Las` is important as we also call out to `Las` and from there to `TyE`.
- This means that all three major components of `LasMetaTyE` can talk to each other and use each other's capabilities.
- With `Las` in the center, we also have access to other mechanisms, for instance fonts, graphics and libraries.
- In `ConTyE` the `Las` language also permits using `int`, `bool`, `char`, `sql` databases and other inputs that can be dealt with programmatically.
- All has been thoroughly optimized for efficiency and performance.

- Transmuting **Mercury** into a library has been a subproduct of the **Luigi2** project. The same official team (Pete, Jack, Helen, Luigi) got John Haddock's blessing.
- It was a rather massive manual (improvised) operation by Pete because multiple number kinds showed up in the metacode and the compiler had to be able to deal with each of them.
- It took all with remaining project (D2C) compatibility.
- The **Mercury** library services both the stand alone programs and **Luigi2**.
- That means the **PostScript** backend had to be built in place using basic (Type1) font handling. We support pdf output via the **Mercury** **haskell** in **MLT** that is used to parsing the **PostScript** and generating the **PDF**.
- In addition, there is **png** and **svg** output. It helps that **Mercury** output is rather simple.
- The **Luigi2** engine uses the **hask** backend which represents the result in **Lex** tables rewriting the **Mercury** source into a representation.
- The library supports decimal and double (internals) but also binary and decimal number models that are linked in libraries.

- We don't need the Postscript backslash (which only does Type1 anyway).
- We also have no use for `arg` and `gout` object.
- The binary number model has no advantages over the decimal one but brings quite some complexity with a library entry.
- The Type1 font feature is not used in ConTeXt but we handle text differently.
- All this means that we can do with a smaller (simplified) MetaPost library.
- The endofiles have been overhauled. We still have a `file` block but use a `Line` script to convert that to C which means that we have better control over how it comes out.
- As with `TeX`, the `file` is handling changing `arg` into `argout` goes to a more integrated.
- The same is true for examining `object` and return values (`object`). That also made for more work in coding.
- Memory management (allocated) is under engine control (as with `TeX` and `LaTeX`) we use a common high performance allocation library.

- Traditional TeX has set the benchmark for typesetting math.
- It uses three categories of fonts: *alphanumeric*, *symbol*, *calligraphic*.
- Limitations in fonts have led to some curious handling of dimensions.
- The fact that there is *never* macros for multispaces *handing and processing*.
- The last family counts (when entering the second pass) so one has to use very controlled font switching.
- Some features (limits, integrals, primes) rely on special macros and parsing.
- One can end-up in tricky font switching. Also, quite some fonts are loaded in order to set up the machinery.

- We now use OpenType math fonts. In fact, in NGLT we always used only one font instance (instead we now have five):
 - We operate in the Unicode domain, so in the end there was no need to change the number of font families in LaTeX.
 - In ConTeXt we now only use a few font families (regular, bold, sl, bold, bold sl).
- All the tricky stuff is done with the help of Lua. This already started very early in the LaTeX project and has not changed.
 - But we still run into issues because the available fonts are inconsistent, incompatible, have a size and that is unlikely to change.
 - We have a virtual font system in place that was used during the transition before, so fonts were available.
 - In ConTeXt there are various ways to deal with shortcomings in, or extensions to, fonts.
- At some point we need to make up our minds, accept the issues with fonts, and just fix things manually.

- In `Lua525` we started with a hybrid approach but eventually ended up with more split code paths.
- In `LuaMetaMIX` nearly all aspects of the engine had made configurable and are under user control.
- This permits experiments where we can apply old methods onto new hosts.

context-2021-overloadprotection

OVERLOAD PROTECTION

the downside of macros

context 2021 meeting

Primitives

A TeX engine comes with a whole set of primitive operations for:

- accessing internal variables
- defining macros
- accessing expansion
- constructing boxes
- building pages
- defining characters (text and math)
- inserting boxes, glue and penalties
- defining fonts
- dealing with language (implementation)
- listing properties
- manipulating tables
- managing macros
- handling marks
- grouping
- subexpressions
- tracing

overload protection — context 2021 meeting — primitives

Macros

- Macros are commands defined by the user under a macro package.
- They can overload a primitive which then can confuse the whole machinery.
- A macro package can also primitives for instance `\relax` can be replaced by `\font\relax after` `\let\let\after` before.
- The only when at definition time the `_` as a letter. By using such a character some primitive against overload is provided.
- In ConTeXt we often, usually abuse like `\internalize` but of course these can also be overloaded.
- We usually overload a vary few primitives, for instance `\language`.
- Users who overload primitives are 'on their own' and 'without support'.
- An easy way to protect yourself is using `\ContextCase` macros.

overload protection — context 2021 meeting — macros

Overload protection

- The `\LoadNoTeX` engine has overload protection built in for the TeX engine as well as provides means to do that for MetaPost.
- In L^AT_EX all commands have been tagged accordingly (which was quite some work).
- Processing a system macro will then do no harm.
- Overload protection is off by default but can be turned on:
`\setboldobjectives{\overloadnoextension}`
`\setboldobjectives{\overloadnoextension}`
- It will always run with the error raised and make sure that the macros obey the rules.
- Modules and/or styles (and in a few cases the more could run cheap and use `\unprotectedloadmodule`):
.....
`\unprotectedloadmodule`

overload protection — context 2021 meeting — overload protection

context-2021-paragraphs

PARAGRAPHS

a bit of an upgrade

context 2021 meeting

Note

- Some of the following already is present for a while and has been discussed at previous meet-ups.
- But... occasionally some minor tweaks you added as consider this to be an update.

paragraphs — context 2021 meeting — note

Spacing

- Spaces in TeX become glue nodes (with optional stretch and shrink).
- In traditional TeX these glue nodes are not extended copies of the current spacing related values.
- In ConTeXt we make real copies so that when we mess with the node list changes to glue don't affect other instances.

paragraphs — context 2021 meeting — spacing

Parameters

- In traditional TeX the paragraphs based properties that are in effect when `\par` happens are used when breaking sections.
- In `\LoadNoTeX` the paragraphs based properties are stored with the paragraph and can be frozen when they are set.
- This gives a more predictable (and robust) way of manipulating a paragraph.
- We can for instance get rid of grouping side effects that interfere with `\overprint`.
- Currently three dozen parameters are tracked but they are grouped in categories.
(show code and examples)

paragraphs — context 2021 meeting — parameters

context-2021-programming

PROGRAMMING

the way ConTeXt is set up

context 2021 meeting

Levels

When you look at ConTeXt bottom-up (engine—interface) you will notice:

1. **primitives**: that is what the engine comes with
2. **infrastructure**: basic management of data structures
3. **helpers**: macros that hide complexity
4. **subsystems**: reflections of macros that implement functionality
5. **mechanisms**: these combine various subsystems
6. **modules**: extra functionality (used 1-5)
7. **styles**: handling sources and layout issues 4-6)

Users normally use ConTeXt top-down (usage—backlog).

programming — context 2021 meeting — levels

Styles

- These are probably solutions for common as well as rare situations.
- The system comes with some styles: the `x-y` files.
- Right from the start the idea was that you get some reasonable default.
- And if you want more you yourself define your style as you go.
- It really is part of the game: exploration.
- Solving your problem is a nice challenge.
- If you want a completely predefined setup, drop somewhere else.

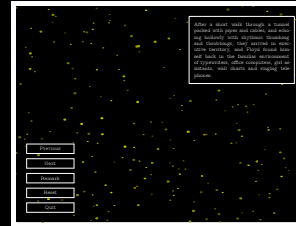
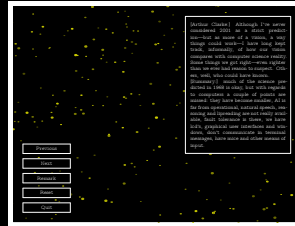
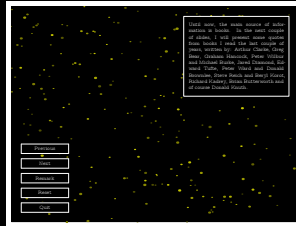
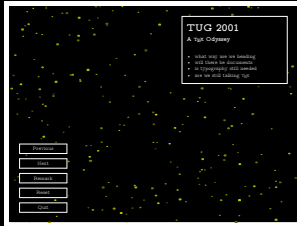
programming — context 2021 meeting — styles

Modules

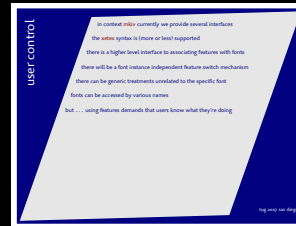
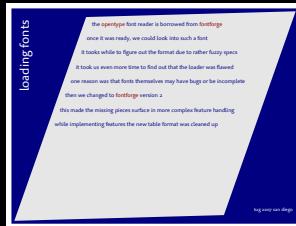
- A lot of functionality is built in.
- This helps to keep the system compact.
- We could cheat and ship thousands of low-level styles but don't do that.
- There are a few mechanisms that don't really fit into the core, so these are implemented as modules that do fit into the interface. See `x-y` and `x-y` files.
- Users can build and share their solutions which has resulted in some third party modules: the `x-y` files.
- For a few, when old people like `TeX` > `many` schemes.

programming — context 2021 meeting — modules

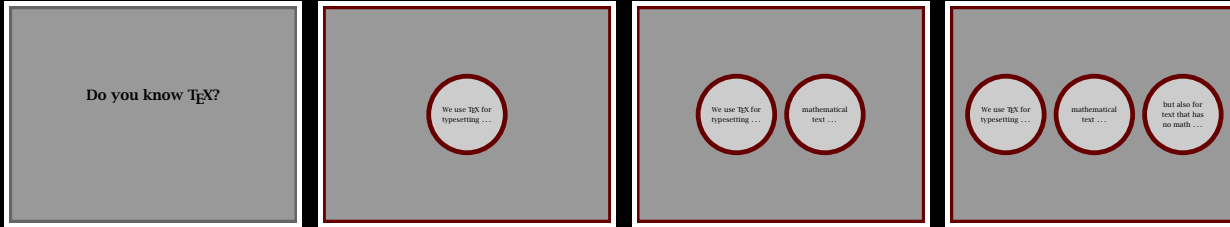
tug-2001-ideas



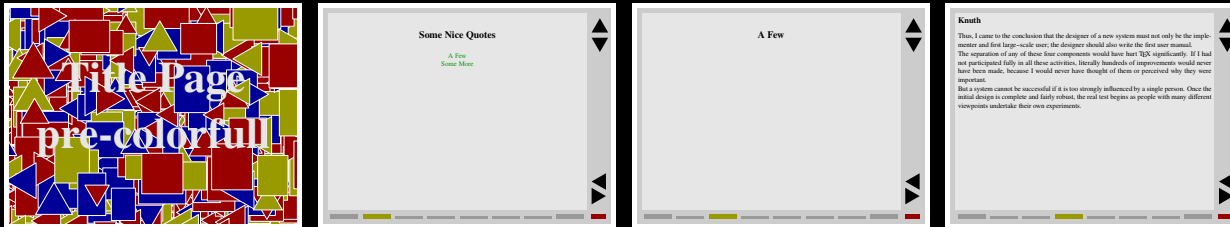
tug-2007-fonts



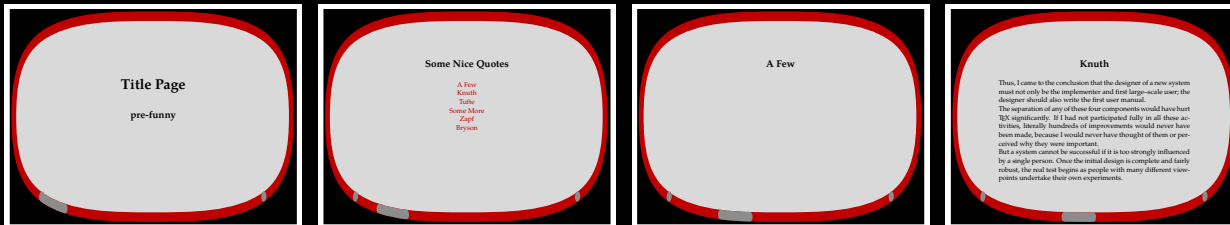
present-balls-001



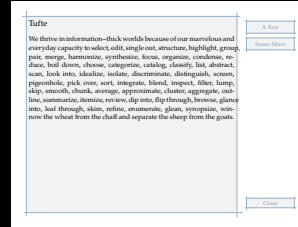
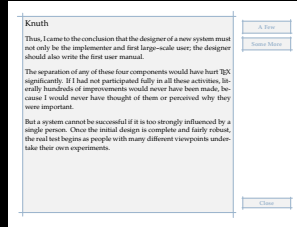
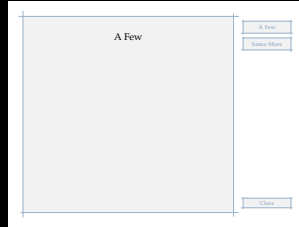
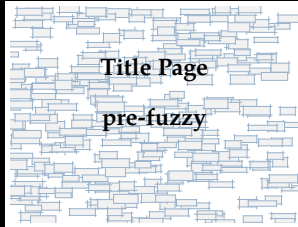
present-colorful-001



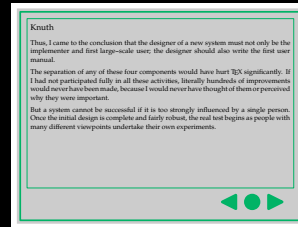
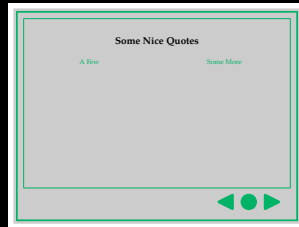
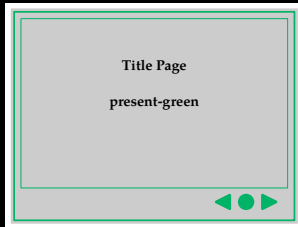
present-funny-001



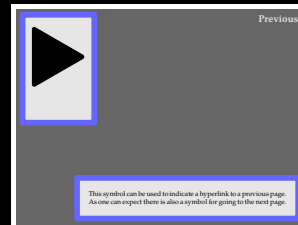
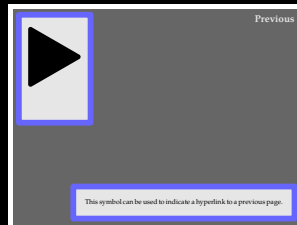
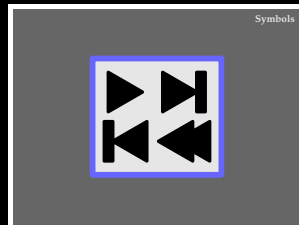
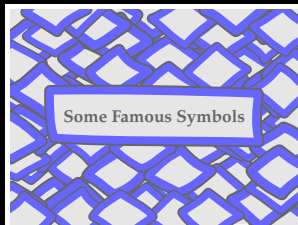
present-fuzzy-001



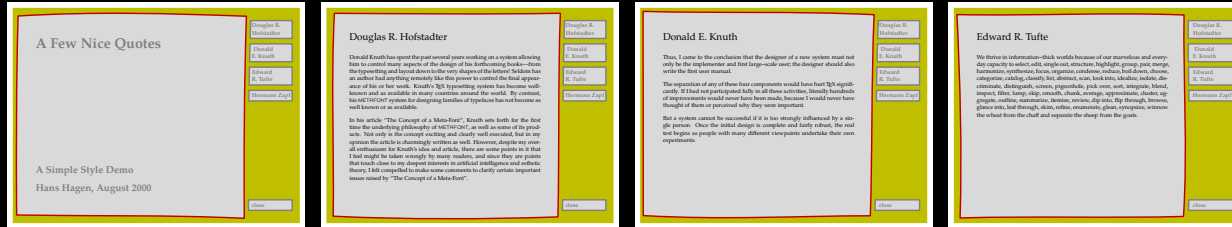
present-green-001



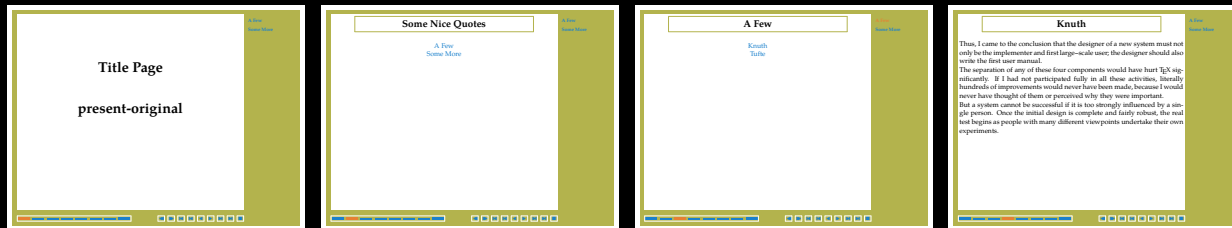
present-grow-001



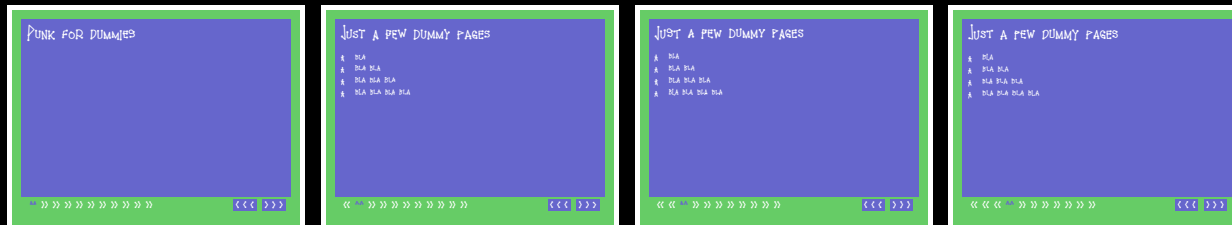
present-organic-001



present-original-001

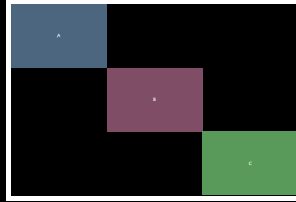
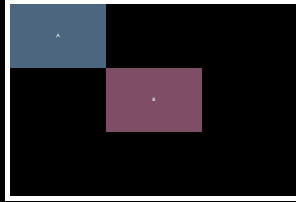


present-punk-001

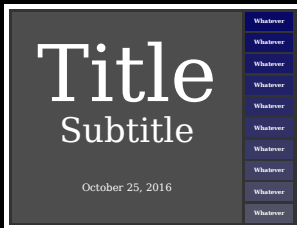


Examples

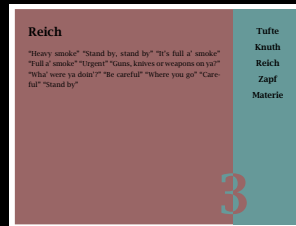
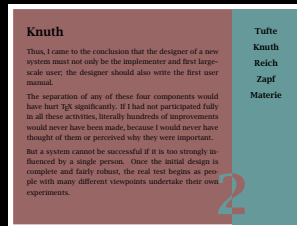
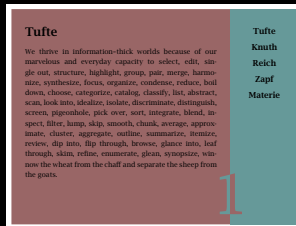
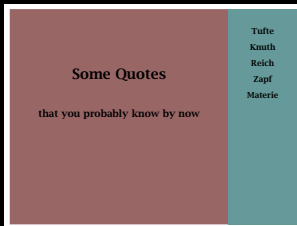
present-random-001



present-shaded-001



present-split-001



present-stepper-001

Stepwise
Refinement

Topics

Female Artists
Male Composers

And Some More

Female Artists

- Fiona Apple
- Tori Amos
- Kate Bush
- Heather Nova
- Alanis Morissette
- Suzanne Vega

Male Composers

- John Adams
- Steve Reich
- Louis Andriessen
- Olivier Messiaen

present-steps-001

Contents

1 Step Set 1 2
2 Step Set 2 1

1 Step Set 1

1 Step Set 1

STEP ONE

1 Step Set 1

STEP ONE

STEP TWO

present-tiles-001

Whatever We
Want Here

Whatever We
Want There

Topic 1	Topic 2	Topic 3
Topic 4	Topic 5	Topic 6
Topic 7	Topic 8	Topic 9
Topic 10	Topic 11	Topic 12

Topic 1

We thrive in information-thick worlds because of our marvelous and everyday capacity to select, edit, single out, structure, highlight, group, put energy, harmonize, synthesize, focus, organize, condense, reduce, boil down, choose, categorize, catalog, classify, list, abstract, scan, look into, identify, isolate, discriminate, distinguish, screen, pigeonhole, pick-over, sort, integrate, blend, inspect, filter, keep, skip, smooth, check, average, approximate, cluster, aggregate, outline, summarize, itemize, review, dip into, flip through, browse, glance into, lead through, skim, refine, enumerate, plan, reorganize, rewire the subject from the chaff and separate the sheep from the goats.

Buttons (roll-over in aacrobath):

click left top home
click right top contents
click left bottom previous
click left bottom next page

Topic 2

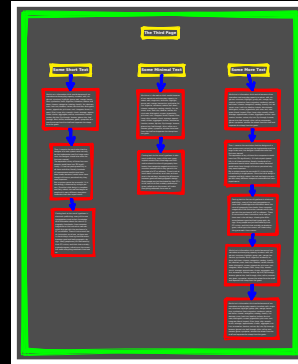
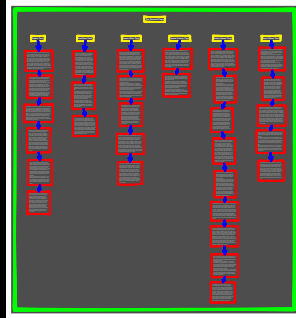
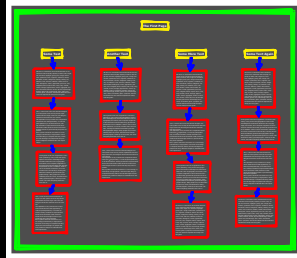
We thrive in information-thick worlds because of our marvelous and everyday capacity to select, edit, single out, structure, highlight, group, put energy, harmonize, synthesize, focus, organize, condense, reduce, boil down, choose, categorize, catalog, classify, list, abstract, scan, look into, identify, isolate, discriminate, distinguish, screen, pigeonhole, pick-over, sort, integrate, blend, inspect, filter, keep, skip, smooth, check, average, approximate, cluster, aggregate, outline, summarize, itemize, review, dip into, flip through, browse, glance into, lead through, skim, refine, enumerate, plan, reorganize, rewire the subject from the chaff and separate the sheep from the goats.

Buttons (roll-over in aacrobath):

click left top home
click right top contents
click left bottom previous
click left bottom next page

Examples

About Text
Today's Talk
Hans Hagen



Quotes, Quotes
and more quotes

[illegible][illegible]

sample: Zaagb, of Shook-ang, Thamee-knight Males, Yano
Wiggen, Banaay-yor, Hoot-ban, Spander-ban, De Khas-ja-
nang, Kung, Nalpa-ban, Yano, Yano-kung, Kikho, Haden,
Naw-ban, Yano, Yano, Yano, Yano, Yano, Yano, Yano,
Yano, ran, Awang, En hwa-ban, En gaw, Yano, Yano, Yano,
Naw-ban, Yano, Yano, Yano, Yano, Yano, Yano, Yano,
Yano, En, Yano, Thamee-ang, Kamp-mah-ban, Shikho-ban,
Kah-yo-yor, Kikho-ban-yor, Yipho-yor-yor, Yipho-yor,
Yano, En, Yano, Thamee-ang, En Kikho, Goff-shah,
Chik, Yano, Haden-ang, Kamp-yipho-ban, En, Haden
Baan, Doot, Dj).